process state to be changed from TASK_RUNNING to either TASK_INTERRUPTIBLE or TASK_UNINTERRUPTIBLE. For this, we have an API *set_current_state()*. Shown below is the modified version of the *read* function:

```
ssize_t read(struct file *filp, char *buff, size_t count,
loff_t *offp)
{
    printk(KERN_INFO "Inside read\n");
    printk(KERN_INFO "Scheduling out\n");
    set_current_state(TASK_INTERRUPTIBLE);
    schedule();
    printk(KERN_INFO "Woken up\n");
    return 0;
}
```

Recompile the program with this change and load the module. Given below is the sample run:

```
$ insmod wait.ko
Major Nr: 250
$ cat /dev/mychar0
Inside open
Inside read
Scheduling out
```

With this, we are able to get the process blocked. The next question is: how do we unblock the process? Since this process is blocked, there has to be some other process which can unblock this. So, does the kernel provide any mechanism to wake up the process? It definitely does! For this, we have an API *wake_up_process()* as shown below:

```
wake_up_process(task_struct *ts)
ts - pointer to task_struct of blocked process
```

Since we have not implemented the *wake up* functionality in the above example, the only way to unblock the process is to signal it. Pressing *Ctrl + c* will terminate the process. So now, let's modify the above program to provide the *wakeup* functionality. Shown below is the modified code snippet:

```
staticstruct task_struct *sleeping_task;

ssize_t read(struct file *filp, char *buff, size_t count,
loff_t *offp)
{
     printk(KERN_INFO "Inside read\n");
     printk(KERN_INFO "Scheduling out\n");
     sleeping_task = current;
     set_current_state(TASK_INTERRUPTIBLE);
     schedule();
     printk(KERN_INFO "Woken up\n");
     return 0;
}

ssize_t write(struct file *filp, constchar *buff, size_t count,
loff_t *offp)
{
     printk(KERN_INFO "Inside Write\n");
     wake_up_process(sleeping_task);
     return count;
}
```

In the above example, we have modified the write function to wake up the blocked process. Since *wake_up_process()* API requires the pointer to the *task_struct* of the blocked process, we update the global variable *sleeping_task* with the task control block of the sleeping process.

Recompile the example with the above changes and load it using *insmod*. Shown below is what we get:

```
$ insmod wait.ko
Major Nr: 250
$ cat /dev/mychar0
Inside open
Inside read
Scheduling out
```

With the current process being blocked in one shell, open another shell and execute the command as shown below:

```
$ echo 1 > /dev/mychar0
Inside open
Inside write
Woken Up
Inside close
Inside close
```

So, here we have two processes – one (cat) blocked on the read call and another (echo) unblocking the first process by invoking the wake-up call. As soon as we execute the echo command, the first process comes out of the waiting mode.

What we have seen here is how to implement the basic wait mechanism in the driver. The process was being put to sleep unconditionally. But in real life scenarios, the process always waits on some valid event. Also, this is more like manual waiting and thereby prone to errors. So, how do we make our process wait on some event? How do we implement a robust wait mechanism in a driver? Does the kernel provide any mechanism for this? To find out the answers to these questions, stay tuned to my next article. Till then, good bye and happy waiting! END

By: Pradeep Tewani

The author works at Intel, Bengaluru. He shares his learning on Linux kernel and embedded systems through his weekend workshops. Learn more about his experiments at *https://sysplay.in*. He can be reached at *pradeep_zenith@hotmail.com*.